

# Real-Time Lane Detection and Autonomous Steering

A comparison of classical pipelines  
and a polynomial-fit improvement

---

David Tovmasyan

Course project, May 2026

# Where we are going

Motivation

The simulator

Three lane detectors

Two controllers

Experiments

Live demo

Discussion and conclusion

# Why lane keeping (still) matters

- Most-deployed AD function: every modern car ships lane-keep assist or adaptive cruise.
- Two coupled problems
  - **Perception:** where is the lane?
  - **Control:** how do we stay in it?
- Learned segmenters dominate benchmarks, but a classical pipeline is:
  - interpretable (debuggable),
  - CPU-friendly ( $> 500$  FPS),
  - zero training data required.



Same frame, three detectors.

## The concrete question

How much performance is left on the table  
by the textbook Canny+Hough detector,  
and can a **polynomial-fit upgrade**  
close the gap on curved tracks  
while staying **real-time**?

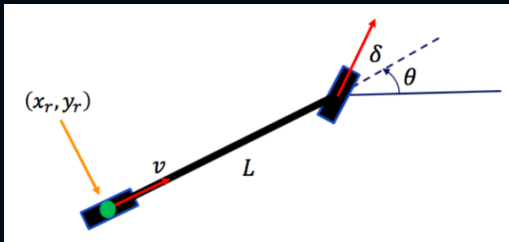
- Implement three classical methods of increasing sophistication.
- Plug each into a PID controller.
- Add a Stanley controller driven from *ground truth* as a perception-free oracle (upper bound on what the controller can do given perfect perception).
- Evaluate on five tracks + a noise sweep.

# What we built

- 2D forward-perspective driving simulator (Python + OpenCV + pygame).
- 5 tracks (difficulty 1–5), closed-loop Catmull–Rom spline.
- Camera: pinhole, 70° FoV, 18° pitch, 640×480 FPV.
- Vehicle: bicycle kinematics,  $\dot{\theta} = (v/L) \tan \delta$ .
- Three live panels: map, FPV overlay, dashboard.



## Vehicle model: bicycle kinematics



Rear-axle reference point  $(x_r, y_r)$ , forward speed  $v$ , wheelbase  $L$ , heading  $\theta$ , front-wheel steering angle  $\delta$ .

Continuous-time kinematics integrated each frame:

$$\dot{x}_r = v \cos \theta$$

$$\dot{y}_r = v \sin \theta$$

$$\dot{\theta} = \frac{v}{L} \tan \delta$$

- Front-wheel steering, rear-axle reference – the standard simplification for lane-keeping work.
- $\delta$  clamped to  $\pm 40^\circ$ ;  $\Delta t$  capped at 50 ms to keep the Euler integration stable.
- No slip, no tire model – realistic enough for highway- speed lane keeping, deliberately simple so the comparison is about perception, not dynamics.

## Ground truth = analytical

Because we own the simulator, we can compute the things a benchmark is normally missing:

- **True lateral offset** from the centreline spline.
- **True heading error** vs. the track tangent.
- **True curvature** ahead (adaptive speed control, turn-sharpness grouping).
- **Ground-truth lane polygon** in the camera frame (re-project road from spline)  $\Rightarrow$  **lane-IoU** against any predicted mask.

$\Rightarrow$  Every metric is anchored to a number the detector *cannot see*.

# Method 1: centroid (our baseline)

## Pipeline

1. Threshold the grey road colour in HSV.
2. Mask to a trapezoidal ROI.
3. Centroid of the blob (image moments).
4. EMA smoothing across frames.

## Role

- Simplest detector we could write  $\Rightarrow$  adopted as the **reference baseline**.
- Tells us how much the lane-line geometry of Methods 2–3 actually buys.

## Spoiler

**Does not complete a lap on any track.**

The blob centroid drifts off-centre whenever the road is asymmetric in the field of view.



## Method 2: Hough lines (the textbook pipeline)

### Pipeline

1. Blur, then Canny edges.
2. Trapezoidal ROI – keep only the road.
3. Probabilistic Hough  $\Rightarrow$  line segments.
4. Split by slope sign: left vs. right lane.
5. Length-weighted average  $\Rightarrow$  one line per side.
6. Lane centre = midpoint at the bottom.

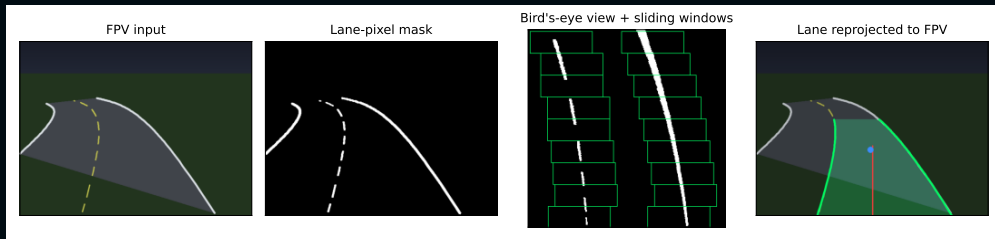
### Strength

- Fast, no training, fully interpretable.
- Rock-solid on straight road.

### Weakness

- **Models every lane as a straight line.**
- In a curve, the line cuts the corner – the error grows with the bend.

## Method 3: bird's-eye view + sliding-window polyfit



Four stages, each fixing a specific weakness of Method 2:

- **Lane pixels** – colour + edges, not just Canny.
- **Bird's-eye warp** – the model sees true road shape.
- **Sliding windows** – robust on dashed lines.
- **Quadratic fit** – a curve, not a straight line.

## Method 3, step 1 – undo the perspective

**Problem.** In the camera image, the same physical dash covers  $\sim 30$  px near the car but only  $\sim 2$  px far ahead. A pixel-counting curve fit therefore listens to the near road and **ignores the far field** – exactly the part that announces upcoming curves.

**Fix.** A fixed homography warps the ROI trapezoid into a square top-down view. In that view:

- Every dash is the same size, near or far.
- Curved lanes become **gentle bends**, not vanishing-point lines.
- A 2nd-order polynomial  $x = ay^2 + by + c$  fits a typical road curve almost exactly.

### Why this is the key step

Without the warp, even a quadratic fit barely beats Hough, because the near road still dominates.

With the warp, near and far pixels get **equal say** in the fit.

## Method 3, step 2 – find each lane

Inside the bird's-eye view, which pixels belong to the left vs. right lane?

### Sliding-window search

1. **Seed**: column histogram of the bottom half  $\Rightarrow$  two strongest peaks = lane bases.
2. **Walk upward**: stack of 9 rectangular windows above each base; collect bright pixels inside.
3. **Recentre**: if enough pixels are found, slide the next window over to follow them. Robust to dashed lines and short gaps.

### Quadratic fit + reprojection

- Least-squares fit  $x = ay^2 + by + c$  on the collected pixels, one polynomial per lane.
- EMA on the coefficients  $\Rightarrow$  stable across frames.
- Warp the curve back to the camera image for the overlay and for the lane-loU metric.

## Method 3, step 3 – the look-ahead offset

- The polynomial gives us the lane *shape*, not just one point on it.
- Evaluate the lane centre at **45 % up the BEV** – a 1-frame predictor, essentially free.
- Feed *that* to the PID instead of the bottom-of- image offset.
- Turn it off and the polyfit's RMS gain shrinks from  $\sim 40\%$  to  $\sim 10\%$ .



The blue dot in the right panel is the reprojected look-ahead point.

## PID – the baseline controller

$$\delta_t = K_p e_t + K_i \sum_{\tau \leq t} e_\tau \Delta t + K_d \dot{e}_t$$

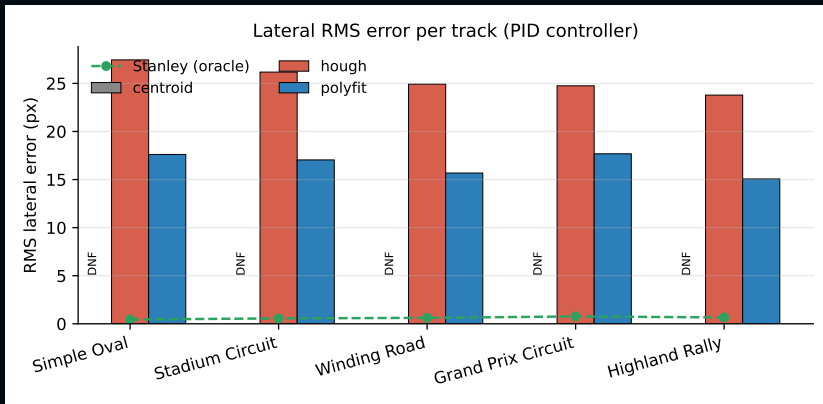
- Input: normalised lateral offset  $e \in [-1, +1]$  from the detector.
- Gains tuned by hand on the oval:  $K_p = 1.2$ ,  $K_i = 0.008$ ,  $K_d = 0.5$ .
- Integral anti-windup at  $\pm 50$ .
- Reactive only – has no model of the path.

## Stanley – the perception-free oracle

$$\delta = \psi + \arctan\left(\frac{k e_{\perp}}{k_s + v}\right)$$

- Heading error  $\psi$  + cross-track error  $e_{\perp}$  at the front axle, both computed from the *true* spline.
- Gains  $k = 0.6$ ,  $k_s = 20$ .
- **It does not look at the camera.**
- Result on every track:  $< 1$  px RMS lateral error.
- Interpretation: the gap between PID-on-detection and Stanley-on-truth is *purely* the perception error.

## Headline result: RMS lateral error per track



Centroid **DNF** on every track. Hough: 25.4 px mean RMS. Polyfit: 16.6 px (-34 %). Stanley+truth: < 1 px.

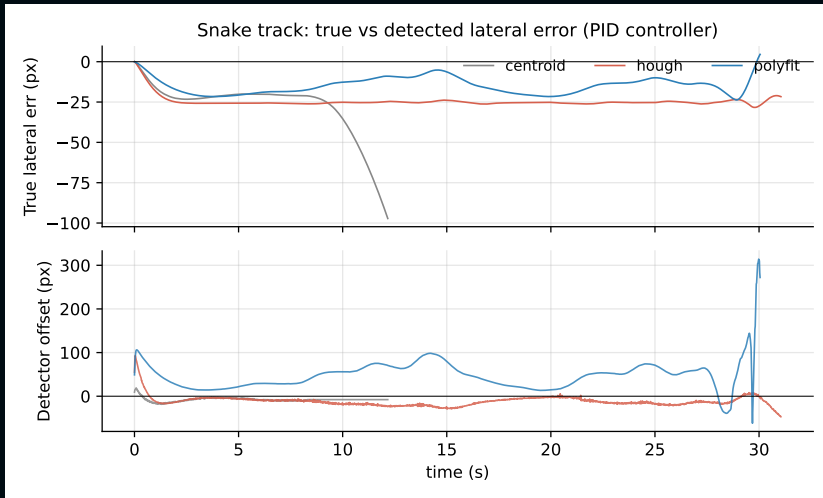


## The full table

| Track              | HOUGH |      | POLYFIT |      | CENTROID | Stanley |
|--------------------|-------|------|---------|------|----------|---------|
|                    | RMS   | IoU  | RMS     | IoU  | RMS      | RMS     |
| Simple Oval        | 27.4  | 0.17 | 17.6    | 0.25 | DNF      | 0.4     |
| Stadium Circuit    | 26.2  | 0.24 | 17.0    | 0.31 | DNF      | 0.6     |
| Winding Road       | 24.9  | 0.23 | 15.7    | 0.35 | DNF      | 0.6     |
| Grand Prix Circuit | 24.8  | 0.26 | 17.7    | 0.36 | DNF      | 0.8     |
| Highland Rally     | 23.8  | 0.31 | 15.1    | 0.39 | DNF      | 0.7     |
| <i>mean (px)</i>   | 25.4  | 0.24 | 16.6    | 0.33 | —        | 0.6     |

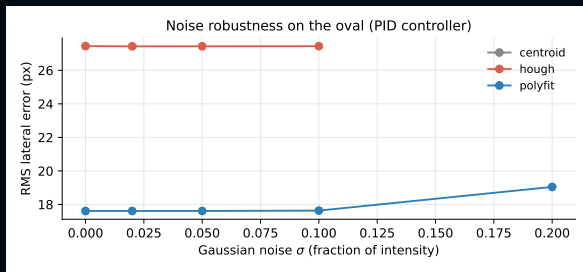
- Polyfit beats Hough on *every* track on both RMS and IoU.
- Lane-IoU rises from 0.24 to 0.33 on average (+**38 %**) – the gain is geometric, not just from the look-ahead.

## What the controller actually sees



**Top:** true error – polyfit bounded, Hough swings  $\pm 30$  px. **Bottom:** detector offset (the PID input). Centroid (grey) leaves the road early.

# Robustness to camera noise



- Same oval, PID, +Gaussian noise  $\sigma \in [0, 0.20]$  per frame.
- Centroid never completes.
- Hough survives  $\sigma \leq 0.10$ , fails at 0.20 (Canny drowned out).
- **Polyfit completes at every  $\sigma$**  – its mask uses a *normalised* Sobel-x + colour.

## Compute cost

| Detector | ms / frame | equiv. FPS |
|----------|------------|------------|
| centroid | 1.7        | ~600       |
| hough    | 0.7        | ~1450      |
| polyfit  | 1.9        | ~520       |

- All three are comfortably real-time on a single CPU thread.
- Polyfit overhead is dominated by the perspective warp and `np.polyfit`, not by the sliding-window scan.
- Plenty of budget left for downstream tasks (other vehicles, obstacles, signs).

```
python main.py --detector polyfit --controller pid
```

- Press **D** to cycle detectors live.
- Press **C** to toggle PID  $\leftrightarrow$  Stanley.
- Press **M** to jump back to the track selector.
- Watch the dashboard: lane offset, FPS, PID components.

Suggested demo route: *Stadium* (Hough wobbles)  $\rightarrow$  *Winding Road* (Hough struggles)  $\rightarrow$  switch to polyfit  $\rightarrow$  switch to Stanley.

# What worked, what failed

## Worked

- Look-ahead offset = single biggest gain.
- HSV + Sobel mask cleaner than Canny on dashed lines.
- Quadratic fits + EMA give stable curves.

## Failed / limitations

- Centroid is structurally biased – no tuning saves it.
- Hough lingers on its last good line through tight bends.
- Polyfit's IPM warp assumes flat ground; real cameras need re-tuning.
- Simulator is too clean – numbers are an upper bound.

## Take-aways

- A modest classical-CV upgrade (BEV warp + sliding-window quadratic fit) cuts RMS lateral error by **34–40 %** vs. the textbook Hough pipeline.
- The change is fully interpretable: we can point at each stage in Fig. 3 and say why it helps.
- Look-ahead offsets are essentially free on top of a polynomial fit and turn a reactive PID into a quasi-MPC.
- Closing the remaining gap to Stanley-on-truth would need sub-pixel-accurate perception  
⇒ a learned segmenter is the next step.

# Thank you.

Code, simulator, report and these slides are all in this repo.

Questions?



## Backup: detector confidence per track

- Centroid mean confidence: **0.27** (HSV blob coverage, near zero whenever the road leaves the trapezoid).
- Hough mean confidence: **0.99** (both lines almost always found – but “found”  $\neq$  “correct”).
- Polyfit mean confidence: **1.00** (both polynomial fits valid in every frame after warm-up).
- Take-away: high detector confidence does not imply low error. Lane-IoU is the better proxy.

## Backup: why the look-ahead works

Recall the bicycle heading update:

$$\dot{\theta} = \frac{v}{L} \tan \delta$$

- A pure-P controller on  $e_{\text{bottom}}$  reacts to *current* position only.
- Adding the look-ahead error  $e_{\text{LA}}$  effectively augments the controller with a free derivative term that knows where the road is going.
- Equivalent to a one-step preview MPC with horizon  $\sim 0.5$  s at typical speeds – without solving an optimisation.

## Backup: reproducibility

```
git clone <repo>
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt matplotlib pdf2image
python experiments.py    # CSVs + summary figures
python make_qualitative_figures.py  # overlays
cd presentation && tectonic report.tex
cd presentation && tectonic slides.tex
```

All experiments are deterministic except the noise sweep, seeded by NumPy's default RNG.